

Architectures de réseaux de neurones dans l'espace des fonctions 1-Lipschitz

Thibaut Boissin

Jury: Edouard Pauwels, Christophe Garcia

Direction: Mathieu Serrurier, Franck Mamalet



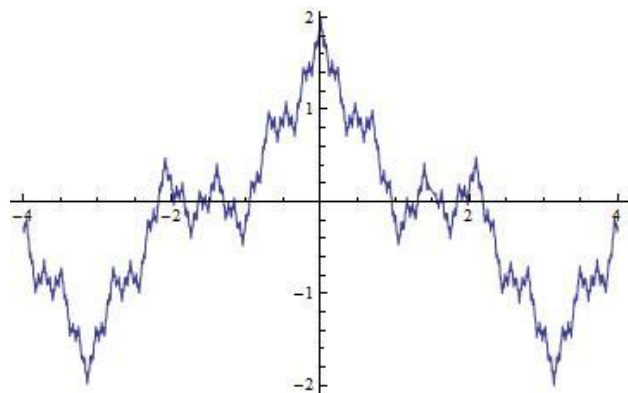
What are 1-Lipschitz networks ?

Lipschitz functions

L-Lipschitz function: there exists **L** such that for all **x, y**

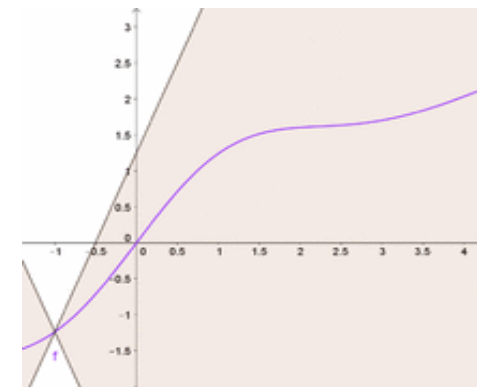
$$\|f(x) - f(y)\| \leq L\|x - y\|$$

Note: **f** is **continuous** and almost everywhere (for Lebesgue measure) differentiable:



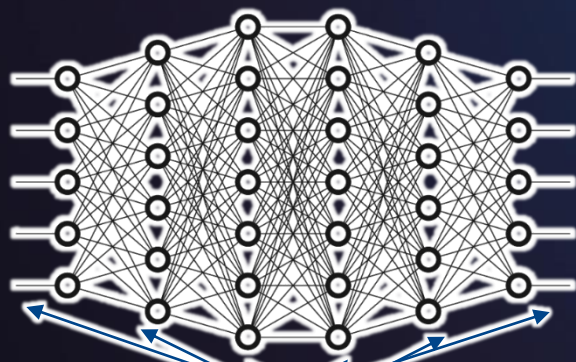
Weierstrass function: continuous but not Lipschitz

$$\|\nabla_x f\| \leq L$$



Lipschitz continuous function

Learning l -Lipschitz functions with NN



$$f(x) = f_6 \circ f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1(x)$$

Weights

- Use **reparametrization** to constrain a layer's weights

Layer

- The layer i with constrained weights only express l_i -Lipschitz functions

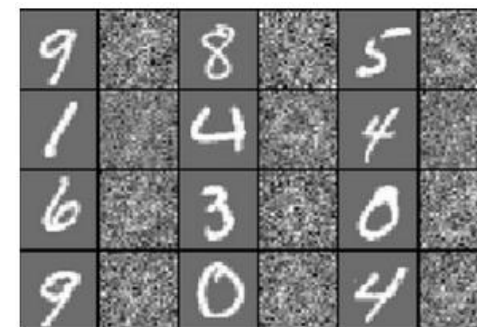
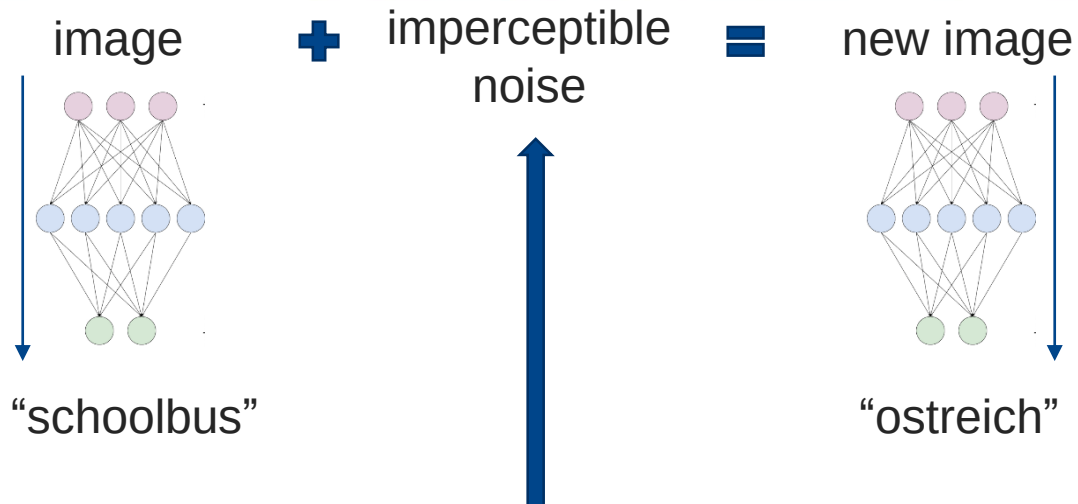
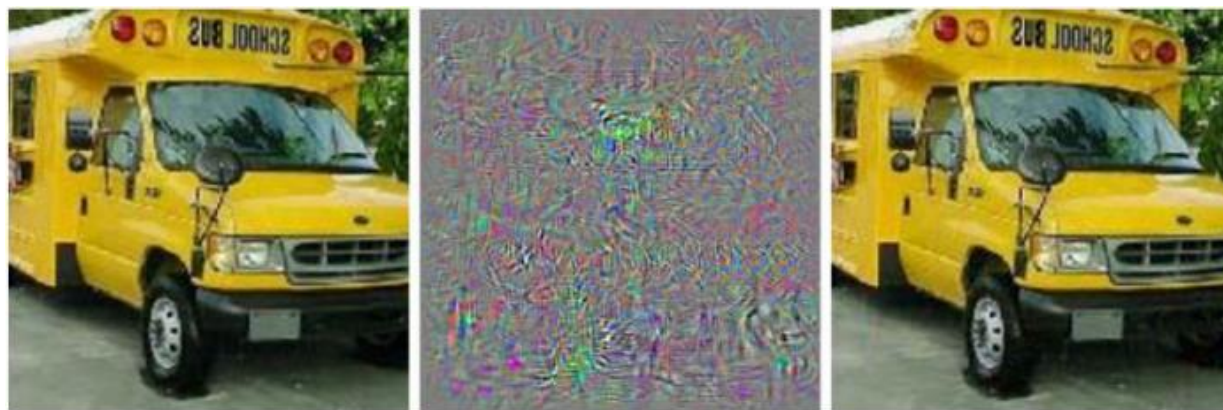
Network

- Lipschitz constant is bounded by the product of each layer's Lipschitz constant

Why 1-Lipschitz networks ?

Evasion attacks (adversarial attack)

Szegedy et al, 2014: “Intriguing properties of neural networks”



Even though the network is robust to random noise!
(right images correctly classified)

Evasion attacks (adversarial attack)

For classification : $l = \operatorname{argmax}(f(x))$

label

neural network

input image

An adversarial attack is:

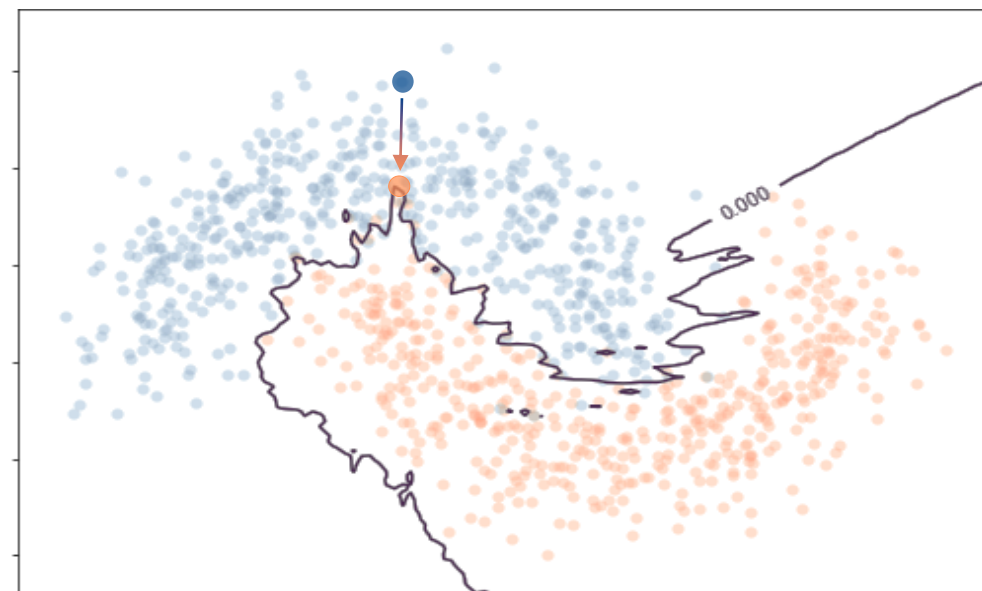
$$\operatorname{adv}(f, x) = \underset{\delta, \operatorname{argmax}(f(x+\delta)) \neq l}{\operatorname{argmin}} \|\delta\|$$

Adversarial noise

Classified differently

$\|\delta\|$

As close as possible to the original one



Certifiable robustness with Lipschitz networks

$$\|\delta\| \geq \|\delta^*\| = \|x - (x + \delta^*)\| \geq \frac{\|f(x) - f(x + \delta^*)\|}{L} \geq \frac{|top_1(f(x)) - top_2(f(x))|}{L\sqrt{2}}$$

Empirical Attacks

The best attack you could ever have (NP-Hard !)

If the network f is L Lipschitz

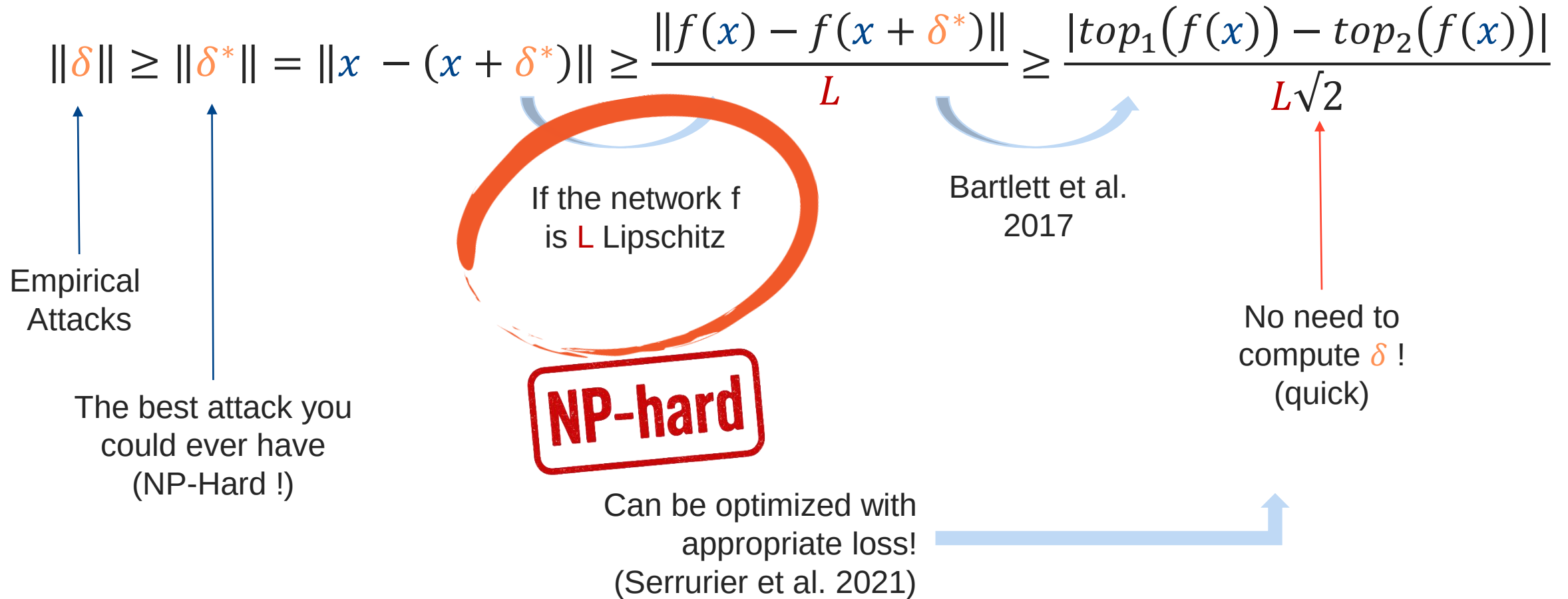
Bartlett et al. 2017

No need to compute δ ! (quick)

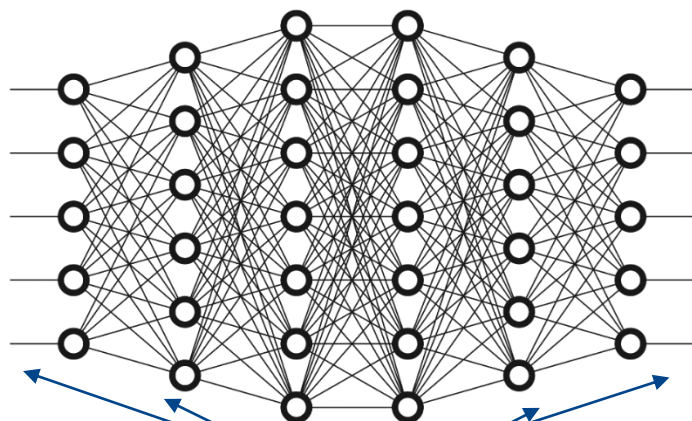
Can be optimized with appropriate loss! (Serrurier et al. 2021)

• **How to build 1-Lipschitz networks ?**

Certifiable robustness with Lipschitz networks



1-Lipschitz networks



$$f(x) = f_6 \circ f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1(x)$$

If you know the Lipschitz constant of each layer you can bound the Lipschitz constant of the whole network:

$$L_f \leq L_{f_6} * L_{f_5} * L_{f_4} * L_{f_3} * L_{f_2} * L_{f_1}$$



NP-hard

If f_i is orthogonal, then $L_{f_i} = 1$ and $L_f \approx \prod L_{f_i}$



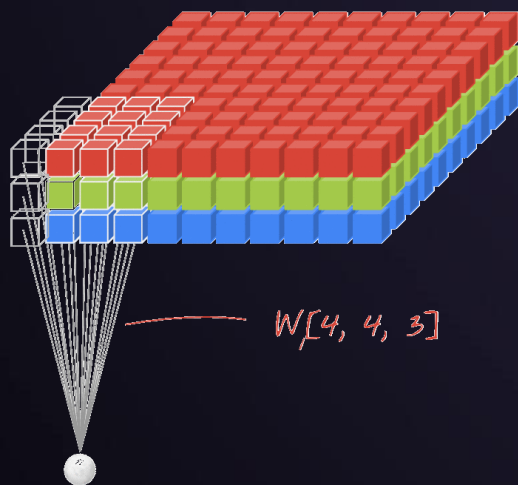
Did we solve an NP-hard problem ?

- Tradeoff tightness/expressiveness
 - Not NP for linear networks
- Intrinsic accuracy/robustness tradeoff [1]
 - implies larger networks & longer training

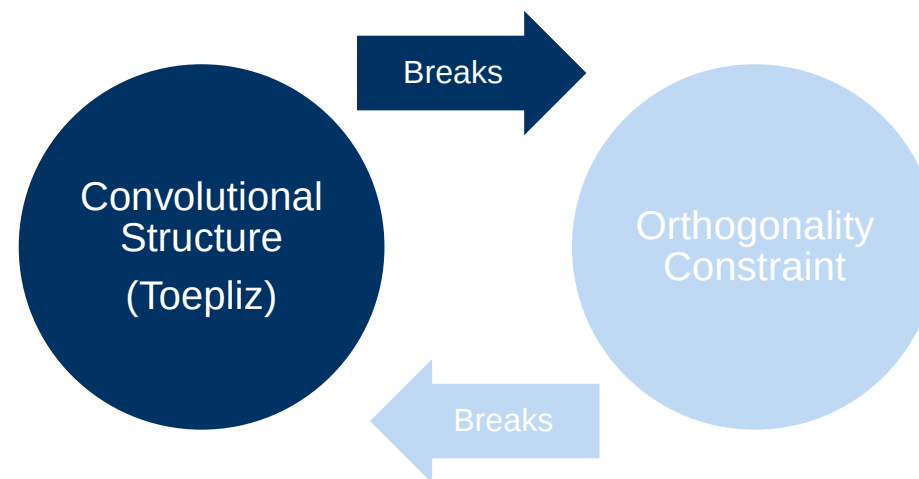
- How to assemble orthogonal blocks ?
- What orthogonal means for:
 - non-linear activations?
 - Convolutions?
 - Attention layers?
 - Other blocks?

Introduction to Orthogonal Convolution

- Linear operator
- Implicit representation of the toepliz matrix



But difficult to build

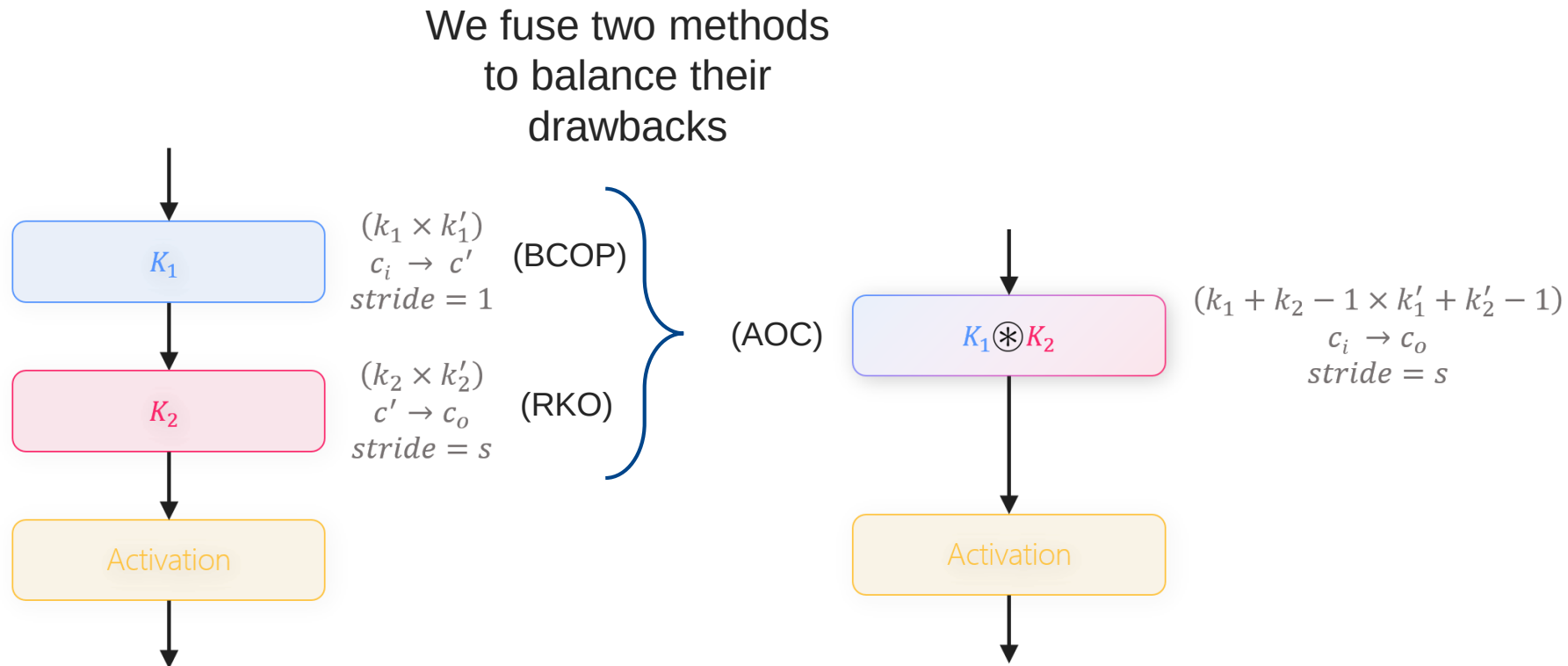


Unlocking Flexibility in Modern Architectures

Method		Orthogonal	Equivalent Kernel Size	Code available	Change Channels	Stride	Conv Transpose	Groups	Dilation
BCOP	(Li et al., 2019)	✓	k	✓	✓	≈	✗	✗	✗
SC-FAC	(Su et al., 2022)	✓	k separable	✗	✓	✓	≈	✓	✓
ECO	(Yu et al., 2022)	✓	$I_w \times I_h$	✗	≈	≈	✗	✗	≈
Cayley	(Trockman & Kolter, 2021)	✓	$I_w \times I_h$	✓	≈	≈	≈	✗	✗
LOT	(Xu et al., 2022)	✓	$I_w \times I_h$	✓	✓	≈	✗	✗	✗
ProjUNN-T	(Kiani et al., 2022)	✓	$I_w \times I_h$	✓	✗	✗	✗	✗	✗
SLL	(Araujo et al., 2023)	✗	composed	✓	✗	✗	✗	✗	✗
Sandwich	(Wang & Manchester, 2023)	✗	composed	✓	≈	≈	✗	✗	✗
AOL	(Prach & Lampert, 2022)	✗	k	✓	✓	✓	✗	✗	✗
SOC	(Singla & Feizi, 2021b)	✓	$k + (n \frac{k}{2})$	✓	≈	≈	✗	✗	✗
AOC	(Ours)	✓	k	✓	✓	✓	✓	✓	✓

This flexibility unlocks their use in modern architectures like U-Net (conv transpose), ResNext (groups), Mobilenetv2 (dilation) and many others

Our approach: AOC



AOC is *fast* and *expressive*

	Models	Acc- uracy	Provable Accuracy $\epsilon = \frac{36}{255}$	Trainable Parameters
CIFAR-10	BCOP	72.2	58.2	2.6M
	GloRo	77.0	58.4	8.0M
	Local-Lip-B	77.4	60.7	2.3M
	Cayley Large	74.6	61.4	21.0M
	SOC 20	78.0	62.7	27.0M
	SOC+ 20	76.3	62.6	27.0M
	CPL XL	78.5	64.4	236.0M
	AOL Large	71.6	64.0	136.0M
	SLL Small	71.2	62.6	41.0M
	SLL Medium	72.2	64.3	78.0M
	SLL Large	72.7	65.0	118.0M
	SLL X-Large	73.3	65.8	236.0M
	Li-Resnets*	82.1	70.1	49.0M
	Li-Resnets++*	87.0	78.1	49.0M
	AOC accurate	91.5	~	2.3M
	AOC robust	74.0	64.3	41.3M
IN-1K	Li-Resnets*	45.6	35.0	86.0M
	Li-Resnets*	49.0	38.3	86.0M
	AOC accurate	68.2	~	53.1M
	AOC robust	42.1	20.3	53.1M

Only 1-Lipschitz
functions here!

Our implementation
makes AOC fast!

Name	Batch Size	Train Time (ms)	Train Memory (GB)	Test Time (ms)	Test Memory (GB)
Conv2D (ref)	128	137 (1.00x)	4.7 (1.00x)	50 (1.00x)	1.4 (1.00x)
AOC (ours)	128	239 (1.75x)	5.3 (1.15x)	53 (1.06x)	1.4 (1.02x)
BCOP	128	389 (2.85x)	8.6 (1.84x)	62 (1.25x)	1.5 (1.06x)
SOC	128	664 (4.86x)	12.8 (2.73x)	429 (8.55x)	1.8 (1.30x)
Cayley	128	584 (4.27x)	19.0 (4.07x)	247 (4.94x)	2.0 (1.45x)
Conv2D (ref)	256	284 (1.00x)	9.1 (1.00x)	91 (1.00x)	2.7 (1.00x)
AOC (ours)	256	354 (1.25x)	9.8 (1.07x)	97 (1.06x)	2.7 (1.01x)
BCOP	256	624 (2.20x)	13.9 (1.53x)	135 (1.48x)	2.8 (1.03x)
Conv2D (ref)	512	550 (1.00x)	17.9 (1.00x)	172 (1.00x)	5.3 (1.00x)
AOC (ours)	512	622 (1.13x)	18.6 (1.04x)	176 (1.02x)	5.4 (1.01x)
BCOP	512	1116 (2.03x)	24.6 (1.38x)	236 (1.48x)	5.4 (1.02x)

Table 3: **AOC benefits from scale.** As demonstrated on a ResNet-34, previous methods impose significant overhead when input images are large (224×224). In contrast, since our method's computational cost is independent of layer input size, its overhead decreases as batch size increases. Furthermore, the low memory overhead enables larger batches at scale.

Beyond AOC: A library with many features



Improved implementations of layers

- ❖ Adaptive-BCOP (AOC)
- ❖ Adaptive-SOC
- ❖ SLL
- ❖ AOL

But also

- ❖ Activations functions
- ❖ Normalization layers
- ❖ Skip connection variants
- ❖ Model factory and zoo (soon !)

Meeting code standards

tests **passing** Pytorch **2.0+** License **MIT**
coverage **94%** Docs **here**



Beyond **Adversarial robustness**

Properties beyond robustness:

- Explainable AI
- Differential Privacy
- Calibration or Conformal guarantees

Useful in other domains:

- Generative modeling
- Long-range RNNs
- Optimizers (Muon/Dion)

What's next ?

Three main lines:

- Pretraining of a Lipschitz backbone
- Exploration of applications to optimizers (Muon/Dion)
- Application to robotics (longer term)

- Publications
 - [An adaptive orthogonal convolution scheme for efficient and flexible cnn architectures](#)
T Boissin, F Mamalet, T Fel, AM Picard, T Massena, M Serrurier
ICML 2025
 - [Orthogonium: A Unified, Efficient Library of Orthogonal and 1-Lipschitz Building Blocks](#)
Thibaut Boissin, Franck Mamalet, Valentin Lafargue, Mathieu Serrurier
ICML 2025 Workshop CODEML
 - [Efficient robust conformal prediction via lipschitz-bounded networks](#)
T Massena, T Boissin, F Mamalet, C Friedrich, M Serrurier, S Gerchinovitz
ICML 2025
- Other
 - PAISS summer school
 - Teaching at AIBT



Dependable, Explainable & Embeddable Learning

Thank you !



What is an orthogonal convolution?

- Orthogonal matrix:

$$WW^T = I \rightarrow \|WW^T - I\|_2 = 0$$

- Convolution:

- Structured matrix with many zeroes and repeated entries
- This matrix is huge !
- Orthogonalization scheme breaks the structure

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

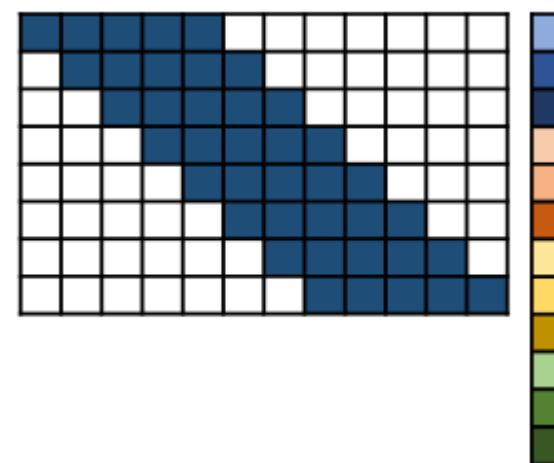
1	0	-1
1	0	-1
1	0	-1

=

6		

$7 \times 1 + 4 \times 1 + 3 \times 1 + 2 \times 0 + 5 \times 0 + 3 \times 0 + 3 \times -1 + 3 \times -1 + 2 \times -1 = 6$

Can be written as a regular
Matrix multiplication



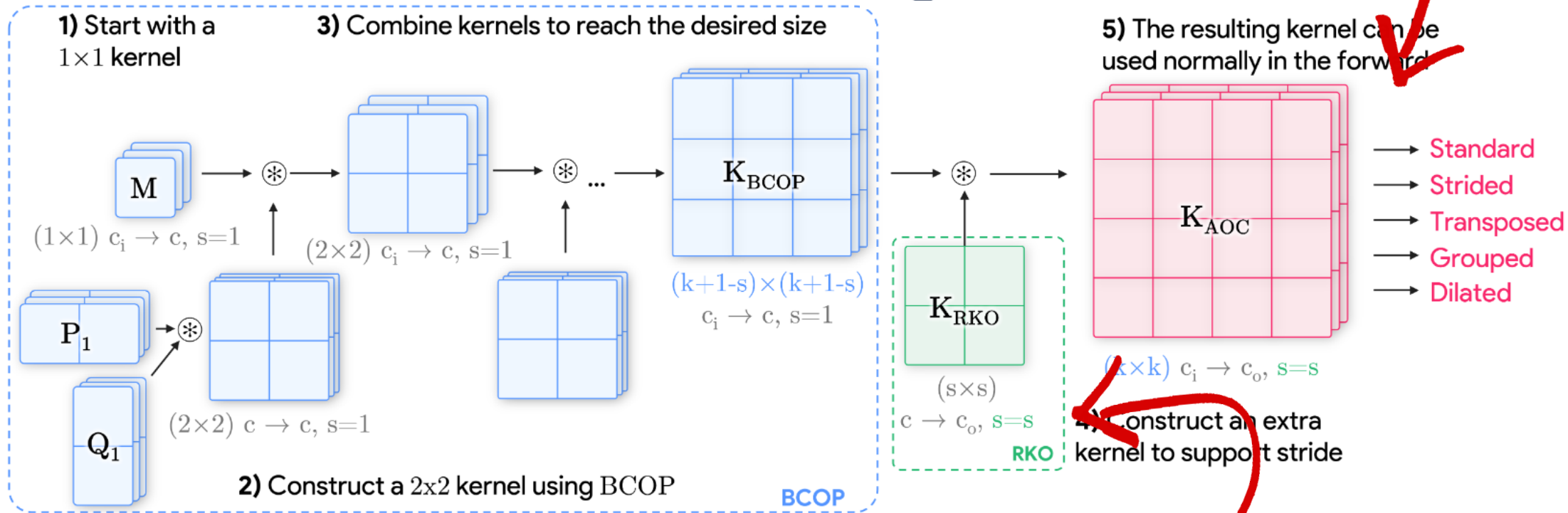
...That unlocks AOC



Handling of native stride



Orthogonal and fast !



Orthogonal



Slow & no native stride

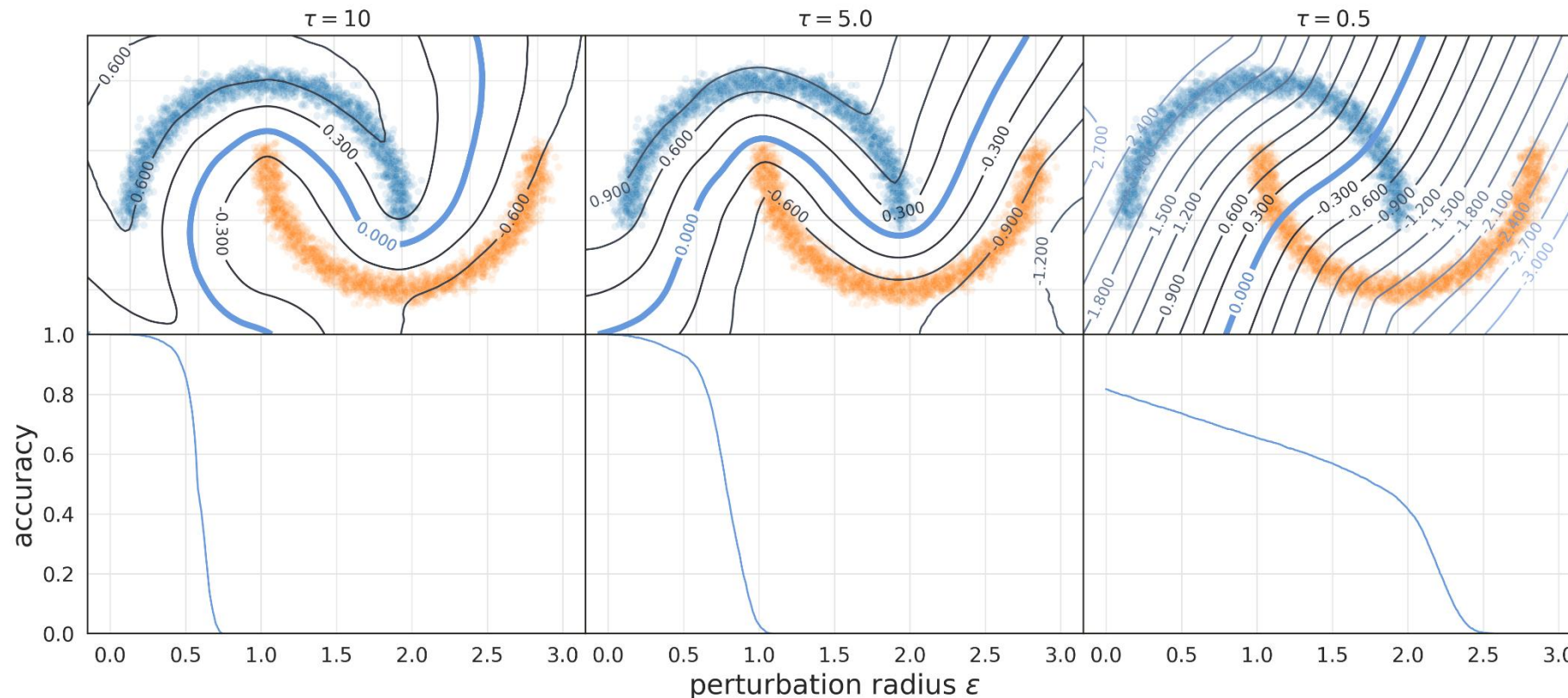


Handling of native stride

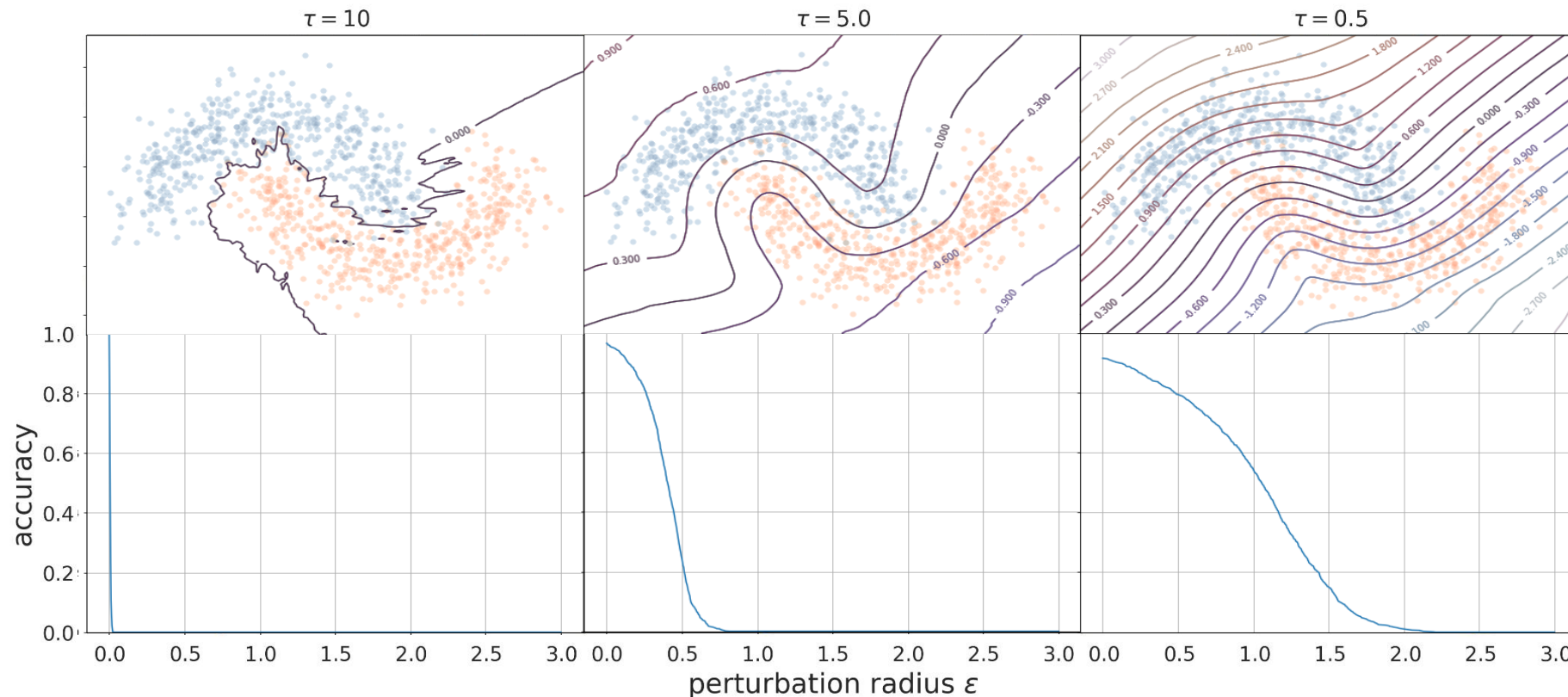


Not orthogonal if $s \neq k$

The accuracy/robustness tradeoff



The accuracy/robustness tradeoff



OPTIMAL TRANSPORT PROPERTIES

Proposition 1 (Transportation plan direction) Let f^* an optimal solution minimizing the $\mathcal{L}_{\lambda, m}^{hKR}$. Given $x \sim \mu$ (resp. ν) and $z = \gamma_\pi(x)$, then $\exists t \geq 0$ (resp. $t \leq 0$) such that $\gamma_\pi(x) = x - t \nabla_x f^*(x)$ almost surely.

Proposition 2 (Decision boundary) Let μ and ν two distributions with disjoint supports with minimal distance ϵ and f^* an optimal solution minimizing the $\mathcal{L}_{\lambda, m}^{hKR}$ with $m < 2\epsilon$. Given $x \sim P$, $x_\delta = x - f^*(x) \nabla_x f^*(x) \in \partial\mathcal{X}$ where $\partial\mathcal{X} = \{x' \in \mathcal{X} | f^*(x') = 0\}$ is the decision boundary (i.e. the 0 level set of f^*).

Link with counterfactual: [DeL21] prove that optimal transport plans can coincide with causal counterfactuals when available.

Prop. 1 demonstrates that gradients of the optimal OTNN solution provide almost surely the direction to the counterfactual

$$\exists t, \gamma_\pi(x) = x - t \cdot \nabla_x f^*(x)$$

